



Instituto Federal de Educação, Ciência e tecnologia Piauí
Curso: Análise de desenvolvimento de sistemas
Disciplina: Banco de Dados 2
Data: 22.06.2026

TRABALHO FINAL

CÓDIGO SQL

```
-- =====
-- 0. RESET (OPCIONAL) - Apaga tudo e recria do zero
-- Use isso se quiser começar limpo
-- =====

DROP DATABASE IF EXISTS floricultura;

CREATE DATABASE floricultura;
USE floricultura;


-- =====
-- 1. CRIAÇÃO DAS TABELAS
-- =====

-- Tabela para os tipos de produto
-- Ex: Flor, Vaso, Adubo
CREATE TABLE tipo_produto (
    codigo INT AUTO_INCREMENT PRIMARY KEY,
    descricao VARCHAR(50) NOT NULL
);

-- Tabela de clientes
CREATE TABLE cliente (
    cpf VARCHAR(14) PRIMARY KEY,
    rg VARCHAR(20),
    nome VARCHAR(100) NOT NULL,
    telefone VARCHAR(20),
    endereco VARCHAR(255)
);

-- Tabela de produtos
CREATE TABLE produto (
    codigo INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
```

```
tipo_codigo INT,  
preco DECIMAL(10,2) NOT NULL,  
quantidade_estoque INT NOT NULL,  
  
-- Chave estrangeira ligando ao tipo de produto  
FOREIGN KEY (tipo_codigo) REFERENCES tipo_produto(codigo)  
);
```

```
-- Tabela da compra (nota fiscal)  
CREATE TABLE compra (  
    numero_nota_fiscal INT AUTO_INCREMENT PRIMARY KEY,  
    cliente_cpf VARCHAR(14),  
    data_compra DATETIME DEFAULT CURRENT_TIMESTAMP,  
    valor_total DECIMAL(10,2) DEFAULT 0.00,
```

```
    -- Compra pertence a um cliente  
    FOREIGN KEY (cliente_cpf) REFERENCES cliente(cpf)  
);
```

```
-- Tabela dos itens de cada compra  
CREATE TABLE itens_compra (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    numero_nota_fiscal INT,  
    produto_codigo INT,  
    quantidade INT NOT NULL,  
    valor_unitario DECIMAL(10,2) NOT NULL,
```

```
    -- Relacionamentos  
    FOREIGN KEY (numero_nota_fiscal) REFERENCES  
compra(numero_nota_fiscal),  
    FOREIGN KEY (produto_codigo) REFERENCES produto(codigo)  
);
```

```
-- =====  
-- 2. STORED FUNCTION
```

```

-- Função para verificar estoque
-- Retorna TRUE se tiver quantidade suficiente
-- =====
DELIMITER //

CREATE FUNCTION fn_checar_estoque(
    p_produto INT,
    p_qtd INT
)
RETURNS BOOLEAN
DETERMINISTIC
BEGIN
    DECLARE v_estoque INT;

    -- Busca quantidade atual em estoque
    SELECT quantidade_estoque
    INTO v_estoque
    FROM produto
    WHERE codigo = p_produto;

    -- Retorna TRUE/FALSE
    RETURN v_estoque >= p_qtd;
END //

DELIMITER ;

-- =====
-- 3. TRIGGER
-- Após inserir item:
-- 1) baixa estoque
-- 2) soma valor total da compra
-- =====
DELIMITER //

CREATE TRIGGER trg_apos_inserir_item

```

```

AFTER INSERT ON itens_compra
FOR EACH ROW
BEGIN
    -- Reduz estoque
    UPDATE produto
    SET quantidade_estoque = quantidade_estoque - NEW.quantidade
    WHERE codigo = NEW.produto_codigo;

    -- Atualiza valor total da compra
    UPDATE compra
    SET valor_total = valor_total + (NEW.quantidade *
NEW.valor_unitario)
    WHERE numero_nota_fiscal = NEW.numero_nota_fiscal;
END //

DELIMITER ;

```

```

-- =====
-- 4. STORED PROCEDURES
-- =====
DELIMITER //

```

```

-- -----
-- Procedure: Criar nova compra
-- Recebe CPF e gera nota fiscal
-- -----
CREATE PROCEDURE sp_nova_compra(
    IN p_cpf VARCHAR(14),
    OUT p_nota_fiscal INT
)
BEGIN
    INSERT INTO compra (cliente_cpf)
    VALUES (p_cpf);

    SET p_nota_fiscal = LAST_INSERT_ID();

```

END //

-- Procedure: Adicionar item na compra

-- Antes verifica se há estoque

CREATE PROCEDURE sp_adicionar_item_compra(

IN p_nota_fiscal INT,

IN p_produto INT,

IN p_quantidade INT

)

BEGIN

DECLARE v_preco DECIMAL(10,2);

-- Verifica estoque

IF fn_checar_estoque(p_produto, p_quantidade) THEN

-- Busca preço atual

SELECT preco

INTO v_preco

FROM produto

WHERE codigo = p_produto;

-- Insere item

INSERT INTO itens_compra (

numero_nota_fiscal,

produto_codigo,

quantidade,

valor_unitario

)

VALUES (

p_nota_fiscal,

p_produto,

p_quantidade,

v_preco

```
);  
  
ELSE  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Erro: Estoque insuficiente.';  
END IF;  
END //
```

```
-- -----  
-- Procedure: Inserir cliente  
-- -----  
  
CREATE PROCEDURE sp_inserir_cliente(  
    IN p_cpf VARCHAR(14),  
    IN p_rg VARCHAR(20),  
    IN p_nome VARCHAR(100),  
    IN p_telefone VARCHAR(20),  
    IN p_endereco VARCHAR(255)  
)  
BEGIN  
    INSERT INTO cliente (  
        cpf,  
        rg,  
        nome,  
        telefone,  
        endereco  
    )  
    VALUES (  
        p_cpf,  
        p_rg,  
        p_nome,  
        p_telefone,  
        p_endereco  
    );  
END //
```

```

-----
-- Procedure: Inserir produto
-----

CREATE PROCEDURE sp_inserir_produto(
    IN p_nome VARCHAR(100),
    IN p_tipo INT,
    IN p_preco DECIMAL(10,2),
    IN p_qtd INT
)
BEGIN
    INSERT INTO produto (
        nome,
        tipo_codigo,
        preco,
        quantidade_estoque
    )
    VALUES (
        p_nome,
        p_tipo,
        p_preco,
        p_qtd
    );
END //

DELIMITER ;

```

```

-- =====
-- 5. DADOS INICIAIS
-- =====

-- Tipos
INSERT INTO tipo_produto (descricao)
VALUES

```



```
('Flor'),  
( 'Vaso'),  
( 'Adubo');
```

```
-- Cliente
```

```
INSERT INTO cliente (  
    cpf,  
    rg,  
    nome,  
    telefone,  
    endereco  
)  
VALUES (  
    '111.222.333-44',  
    '1234567',  
    'João Silva',  
    '869999999999',  
    'Rua A, 123'  
);
```

```
-- Produtos
```

```
INSERT INTO produto (  
    nome,  
    tipo_codigo,  
    preco,  
    quantidade_estoque  
)  
VALUES  
( 'Rosa Vermelha', 1, 5.50, 50),  
( 'Vaso de Cerâmica M', 2, 35.00, 10);
```

```
-- =====  
-- 6. CONSULTAS PARA TESTAR  
-- =====
```

-- Ver clientes

SELECT * FROM cliente;

-- Ver produtos

SELECT * FROM produto;

-- Ver compras

SELECT * FROM compra;

-- Ver itens da compra

SELECT * FROM itens_compra;